Apuntes - capítulo 1

resumen general del libro

Capítulo 1

Internet

Es una red de computadoras específica, una infraestructura que proporciona servicios a las aplicaciones. Se dice que las aplicaciones son **aplicaciones distribuidas**, porque implican a varios sistemas terminales que intercambian datos entre sí. **Las aplicaciones de Internet se ejecutan sobre sistemas terminales.** Aunque los dispositivos de conmutación de paquetes facilitan el intercambio de datos entre sistemas terminales, no se preocupan si la aplicación que está actuando como origen o destino de los datos.

Los sistemas terminales conectados a internet proporcionan una **API (Application Programming Interface)**, que especifica cómo un programa de software que se ejecuta en un sistema terminal pide a la infraestructura de Internet que suministre datos a un programa de software de destino específico que se ejecuta en otro sistema terminal.

Componentes de internet

Internet es una red de computadoras que interconecta cientos de millones de dispositivos informáticos a lo largo del mundo. Estos dispositivos se denominan ***hosts* o sistemas terminales** (su nombre viene de que se sitúan en la frontera de la red). Los sistemas terminales se conectan entre sí mediante una red de **enlaces de comunicaciones** y dispositivos de conmutación de paquetes. Los distintos enlaces de comunicaciones pueden transmitir los datos a distintas velocidades. La **velocidad de transmisión de un enlace** se mide en bits/segundo. Cuando un sistema quiere enviar datos a otro sistema terminal, el emisor segmenta los datos y añade bits de cabecera a cada segmento. Los **paquetes** de información resultantes, se envían a través de la red hasta el sistema terminal receptor, donde vuelven a ser ensamblados para obtener los datos originales.

Un **conmutador de paquetes** toma el paquete que llega de uno de sus enlaces de comunicaciones de entrada y lo reenvía a uno de sus enlaces de comunicaciones de salida. Los dispositivos de conmutación de paquetes son variados, pero los que más se utilizan son los **router y los switches de la capa de enlace**. Ambos tipos reenvían los paquetes hacia sus destinos finales. Los switches de la capa de enlace normalmente se emplean en las redes de acceso, mientras que los routers suelen utilizarse en el núcleo de la red. La **ruta** es la secuencia de enlaces de comunicaciones y conmutadores de paquetes que atraviesa un paquete desde el sistema terminal emisor hasta el sistema terminal receptor.

Los sistemas terminales acceden a Internet a través de los **ISP** (Internet Service Provider). Cada ISP es en sí mismo una red de conmutadores de paquetes y enlaces de comunicaciones. Los ISP proporcionan una amplia variedad de tipos de acceso a red a los sistemas terminales. Estos, también proporcionan acceso a Internet a los proveedores de contenido, conectando sitios web directamente a Internet. Internet es todo lo que conecta a los sistemas terminales entre sí, por lo que los ISP que proporcionan el acceso a los sistemas terminales también tienen que estar conectados entre ellos. Los ISP de nivel

inferior se conectan con los ISP de nivel superior nacionales e internacionales. La red de cada ISP se administra de forma independiente, ejecutando el protocolo IP y se ajusta a determinados convenios de denominación y de asignación de direcciones.

Los sistemas terminales, los conmutadores de paquetes y otros dispositivos de Internet ejecutan **protocolos** que controlan el envío y la recepción de información dentro de Internet. El protocolo **TCP** (Transmission Control Protocol) y el protocolo **IP(Internet Protocol)**, son dos de los protocolos más importantes de Internet y se conocen como **TCP/IP**. El protocolo IP especifica el formato de los paquetes que se envían y se reciben entre los routers y sistemas terminales.

Los estándares de Internet son desarrollados por el IETF, y los documentos asociados al mismo se conocen como **documentos RFC** (Request For Comments). Estos documentos definen los protocolos y se utilizan para solucionar los problemas de diseño de la red y de los protocolos.

Protocolos

Es necesario que las entidades (componentes del HW o SW) que deseen comunicarse ejecuten el mismo protocolo para poder llevar a cabo la tarea. Está dado por la transmisión y recepción de mensajes, y las acciones específicas que tomamos como respuesta a los mensajes de contestación recibidos o a otros sucesos. **Cualquier actividad de Internet que implique que dos entidades remontan se comuniquen, es un protocolo**. En resumen, un protocolo define el formato y el orden de los mensajes intercambiados entre dos o más entidades que se comunican, así como las acciones tomadas en la transmisión y/o recepción de un mensaje u otro suceso.

Frontera de la red

Los host albergan programas de aplicación (como navegadores web, servidores web, programas de lectura de mensajes de correo electrónico o servidores de correo electrónico). Estos host se clasifican en dos categorías: **clientes y servidores.** Un programa cliente es un programa que se ejecuta en un sistema terminal que solicita y recibe de un programa servidor que se ejecuta en otro sistema terminal. Se dice que son aplicaciones distribuídas porque, por lo general, el programa cliente se ejecuta en un sistema terminal y el programa servidor en otro. Estos dos programas interactúan enviándose mensajes a través de Internet. Actualmente hay muchos programas P2P o Peer-to-Peer, donde los sistemas terminales interactúan y ejecutan programas que realizan tanto funciones de cliente como de servidor.

Redes de acceso

Son los enlaces físicos que conectan un sistema terminal con el primer router, o router de frontera, de una ruta entre el sistema terminal y otro sistema terminal distante.

Un **telco** local es una infraestructura de la telefonía cableada tradicional local. Cada residencia dispone de un enlace directo de cobre de par trenzado a un switch de la compañía, el cual está conectado a la central telefónica. Se enlazan los clientes con la central más próxima.

Los sistemas terminales se conectan entre sí mediante una red de enlaces de comunicaciones y conmutadores de paquetes. Los distintos enlaces pueden transmitir los datos a diferente velocidad y la velocidad de transmisión de un enlace de mide en bits/segundo. Cuando un sistema terminal tiene datos que enviar a otro sistema terminal, el emisor segmenta los datos y añade bytes de cabecera a cada segmento. LOs paquetes de información resultantes, conocidos como paquetes, se envían a través de la red hasta el sistema terminal receptor, donde vuelven a ser ensamblados para obtener los datos originales.

Un conmutador de paquetes toma un paquete que llega a través de uno de sus enlaces de comunicación de entrada y lo reenvía a través de uno de sus enlaces de comunicaciones de salida.

Los ISP de nivel inferior se interconectan a través de ISP de nivel superior nacionales e internacionales. Un ISP de nivel superior está compuesto por routers de alta velocidad interconectados a través de enlaces de fibra óptiima de alta velocidad. La red de cada ISP se administra de forma independiente, ejecuta el protocolo IP y se ajusta a determinados convenios de denominación y de asignación de direcciones.

FTC QUEDÓ DESCONTINUADO
TCP IP ES LO QUE MÁS SE USA
HTTP FUNCIONA EN TCP HASTA LA 3.0
UDP se usa para hacer streaming

HTTP ES UN PROTOCOLO Y QUICK ES UN AGREGADO ENTR ELA CAPA DE TRANSPORTE Y LA DE APLICACIÓN, CAPA INTERMEDIA PARA HACER QUE UDP SEA ÚTIL. Con quick le dan seguridad entonces no vas a perder paquetes O TE VAN A AVISA.

Las aplicaciones se ejecutan sobre los sistemas terminales. Los sistemas terminales conectados a Internet proporcionan una API (Application Programming Interface) que especifica cómo un programa de software que se ejecuta en un sistema terminal pide a la infraestructura de internet que suministre datos a un programa de software de destino específico que se ejecuta en otro sistema terminal.

Capítulo 2 - HTTP

Capítulo 2 - HTTP

Las aplicaciones de red son la razón de ser de una red de computadoras, ya que si la aplicación no sirve, no tiene sentido diseñar protocolos de red para darle soporte a las mismas.

Capa de aplicación

El desarrollo de una aplicación de red implica escribir programas que se ejecuten en distintos sistemas terminales y que se comuniquen entre sí a través de la red.

Si el programa es P2P, es decir que cada host actúa como proceso cliente y proceso servidor a la vez, los programas instalados en los distintos host pueden ser similares o idénticos. Esto supone que al desarrollar el SW, este se ejecutará en varios sistemas. **No es necesario escribir SW que se ejecute en los dispositivos del núcleo de la red,** ya que estos no operan en la capa de aplicación, sino en las capas inferiores (capa de red e inferiores).

Arquitectura de las aplicaciones de red

La arquitectura de una aplicación es muy distinta a la arquitectura de la red. Esta última, es fija y proporciona un conjunto específico de servicios a las aplicaciones, mientras que la arquitectura de una aplicaciones establece cómo la aplicación debe estructurarse en los distintos sistemas terminales. Al seleccionar una arquitectura de la aplicación el desarrollador probablemente utilizará uno de los paradigmas arquitectónicos predominantes en las aplicaciones de red modernas: cliente-servidor o P2P.

La comunicación de una aplicación de red tiene lugar entre sistemas terminales de la capa de aplicación.

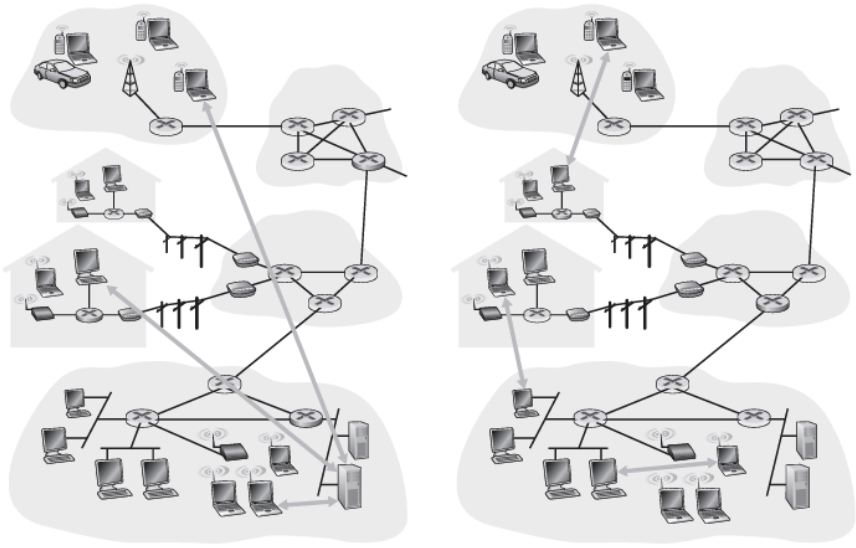
En una **arquitectura cliente-servidor** siempre existe un host activo, denominado servidor, que da servicio a las solicitudes de otros host, que son los clientes. Los clientes pueden estar siempre activos o de forma intermitente. Con esta arquitectura, los clientes no se comunican directamente entre sí. Otra característica de este modelo es que el servidor tiene una dirección fija y conocida (por lo que siempre está activo), denominada dirección IP. Un cliente siempre puede contactar con él enviando un paquete a su dirección. Normalmente, un único host servidor es incapaz de responder todas las solicitudes de sus clientes, por lo que en esta arquitectura suele utilizarse una agrupación (cluster) de host, llamado **centro de datos**, para crear un servidor virtual de gran capacidad. Los servicios de aplicaciones basadas en esto, suelen precisar una arquitectura intensiva porque requieren que los proveedores de servicios compren, instalen y mantengan granjas de servidores. Además del coste de las recurrentes interconexiones y ancho de banda para enviar datos a Internet y recibirlos.

En la **arquitectura P2P** existe una mínima dependencia de una infraestructura de servidores siempre activos. En su lugar, la aplicación explota la comunicación directa entre parejas de host conectados de forma intermitente, conocidos como peers. Esta arquitectura se utiliza en muchas de las aplicaciones actuales más populares. Esta arquitectura es auto-escalable y una buena relación coste-prestaciones porque no suelen requerir una

infraestructura de servidores significativa ni gran ancho de banda de servidor. Poseen tres problemas principales:

1. Orientadas al ISP: la mayoría de los ISP residenciales están dimensionados para hacer un uso “asimétrico” del ancho de banda (dar más soporte de descarga que de carga).
2. Seguridad: en peligro por su naturaleza extremadamente distribuída y abierta.
3. Incentivos: el éxito de las aplicaciones P2P depende de convencer a los usuarios para ofrecen voluntariamente a las aplicaciones recursos de ancho de banda, de almacenamiento y de computación.

Existen aplicaciones con **arquitecturas híbridas** que combinan elementos cliente-servidor y P2P. Por ejemplo, en aplicaciones de mensajería instantánea los servidores se utilizan para hacer un seguimiento de las direcciones IP de los usuarios, pero los mensajes de usuario a usuario se envían directamente entre los hosts.



a. Arquitectura cliente-servidor b. Arquitectura P2P

seguir p 86

Formato de los mensajes HTTP

página 101

La especificación de HTTP define un gran número de líneas de cabecera que pueden ser insertadas por los navegadores, servidores web y servidores de almacenamiento en caché de la red. Un navegador generará líneas de cabecera en función del tipo y la versión del navegador, de la configuración del navegador que tenga el usuario y de si el navegador actualmente tiene en caché una versión del objeto, posiblemente desactualizada. Los servidores web se comportan de forma similar: existen productos, versiones y configuraciones diferentes, que se incluyen en las líneas de cabecera que se incluirán en los mensajes de respuesta. EJEMPLOS:

REQUERIMIENTOS

GET /unadireccion/pagina.html HTTP/1.1

Host: www.unaescuela.edu

Connection: close

User-agent: Mozilla/4.0

Accept-language: fr

La primera línea se denomina **línea de solicitud** y las siguientes son **líneas de cabecera**. La línea de solicitud tiene tres campos: el campo de método (GET, POST, HEAD, PUT, DELETE), el campo URL y el campo de la versión HTTP. La mayoría de los mensajes de solicitud HTTP usan el método GET. Este método se emplea cuando el navegador solicita un objeto, identificando dicho objeto en el campo URL.

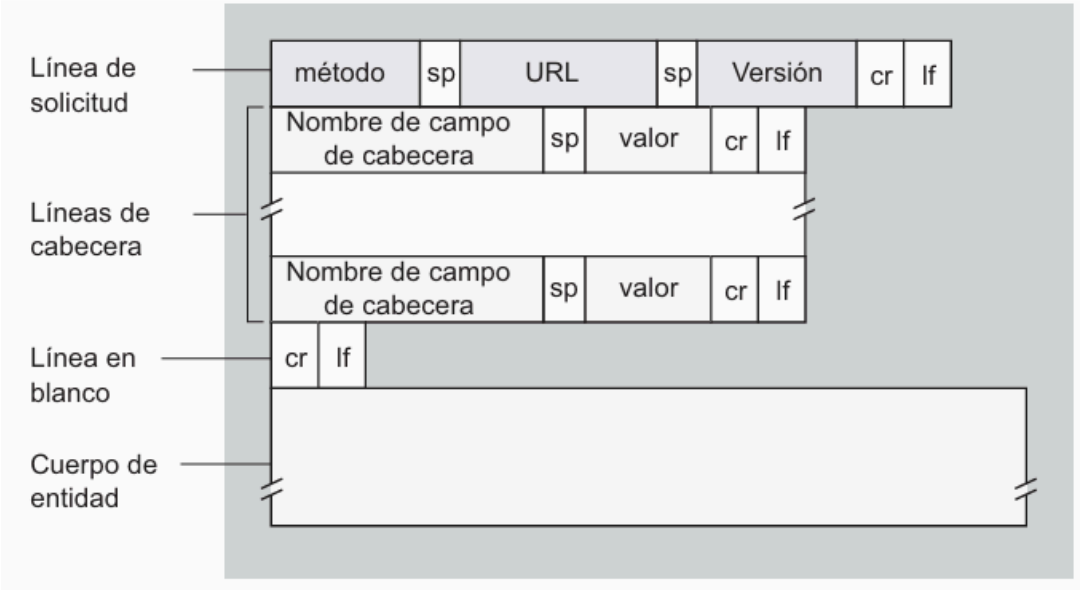
En las líneas de cabecera tenemos

- Host: especifica el host en el que reside el objeto. Esta línea es necesaria porque las cachés proxy web necesitan la información proporcionada por la línea de cabecera del host.
- Connection: close - el navegador está diciendo al servidor que no desea molestarse en trabajar con conexiones persistentes, sino que desea que el servidor cierre la conexión después de enviar el objeto solicitado.
- User-agent: especifica el agente de usuario, es decir, el tipo de navegador que está haciendo la solicitud al servidor. Esto es útil porque el servidor puede enviar versiones diferentes del mismo objeto a los distintos tipos de agentes de usuario (el mismo URL direcciona a cada una de las versiones).
- Accept-language -> es una de las cabeceras de negociación del contenido disponibles en HTTP
- Cuerpo de entidad: este campo queda vacío cuando se utiliza el método GET, pero no cuando se usa el método POST. Un cliente HTTP suele usar el método POST cuando el usuario completa un formulario. Con el mensaje post, el usuario solicita una página web al servidor, pero el contenido concreto de la misma dependerá de lo que el usuario haya escrito en los campos del formulario. Este cuerpo de entidad contendrá lo que el usuario haya introducido en los campos del formulario. A veces los formularios HTML usan el método GET e incluyen los datos de entrada, especificados en los campos del formulario, en la URL solicitada. Por ejemplo, si un formulario emplea el método GET y tiene dos campos, y las entradas a esos dos campos son monkeys y bananas, entonces el URL tendrá la estructura www.unsitio.com/animalsearch?monkeys&bananas.

  El método HEAD es similar al GET. Cuando el servidor recibe una solicitud con el método HEAD, responde con un mensaje HTTP pero excluye el objeto solicitado. Se suele usar para labores de depuración.

  El método PUT se usa junto con herramientas de publicación WEB. Esto permite a un usuario cargar un objeto en la ruta específica (directorio) en un servidor web determinado. También lo usan las aplicaciones que necesitan cargar objetos en servidores web.

  El método DELETE permite a un usuario o a una aplicación borrar un objeto de un servidor web.



RESPUESTA

página 103

HTTP/1.1 200 OK

Connection: close

Date: Sat, 07 Jul 2007 12:00:15 GMT

Server: Apache/1.3.0 (Unix)

Last-Modified: Sun, 6 May 2007 09:23:24 GMT

Content-Length: 6821

Content-Type: text/html

(datos datos datos datos datos ...)

Tiene tres secciones: una línea de estado inicial, líneas de cabecera y el cuerpo de entidad. El cuerpo de entidad es la parte más importante del mensaje, ya que contiene el objeto solicitado en sí. La línea de estado contiene tres campos: el que especifica la versión del protocolo, el correspondiente al código de estado y el tercero que contiene el mensaje explicativo del estado correspondiente.

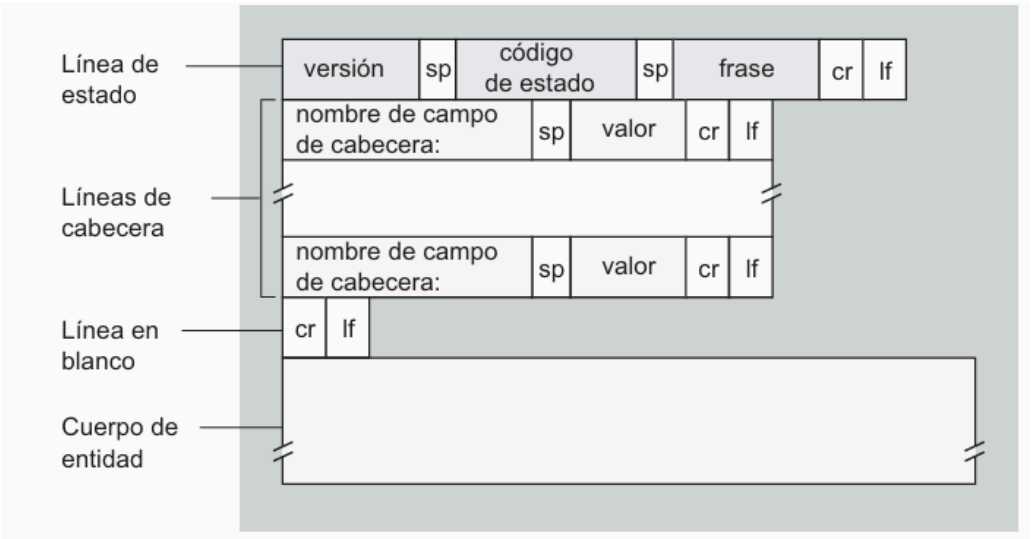
En las líneas de cabecera, el servidor utiliza:

- Connection: close para indicar que el cliente va a cerrar la conexión TCP después de enviar el mensaje
- Date: indica la hora y la fecha en la que se creó la respuesta HTTP y fue enviada por el servidor (no especifica cuando el objeto fue creado o modificado por última vez). Es la hora en la que el servidor recupera el objeto de su sistema de archivos, inserta el objeto en el mensaje de respuesta y lo envía.

- Server: indica que el mensaje fue generado por un servidor web Apache. Es la analogía a la cabecera User-agent.
- Last-modified: especifica la hora y fecha que el objeto fue creado o modificado por última vez. Esta línea es fundamental para el almacenamiento en caché del objeto, tanto en el cliente local como en los servidores de almacenamiento en caché de la red.
- Content-Length: especficia el número de bytes del objeto que está siendo enviado
- Content-Type: indica que el objeto especificado en el cuerpo de entidad es texto HTML (supone que el tipo se especifica acá y no por la extensión del archivo).

El código de estado y su frase asociada indican el resultado de la solicitud. Algunos códigos de estado y su frase asociada son:

- 200 OK: La solicitud se ha ejecutado con éxito y se ha devuelto la información en el mensaje de respuesta.
- 301 Moved Permanently: El objeto solicitado ha sido movido de forma permanente; el nuevo URL se especifica en la línea de cabecera Location: del mensaje de respuesta. El software cliente recuperará automáticamente el nuevo URL.
- 400 Bad Request: Se trata de un código de error genérico que indica que la solicitud no ha sido comprendida por el servidor.
- 404 Not Found: El documento solicitado no existe en este servidor.
- 505 HTTP Version Not Supported: La versión de protocolo HTTP solicitada no es soportada por el servidor.



seguir pag 105 - cookies

Práctica 1

Redes y Comunicaciones LINTI - UNLP Práctica 1 - Introducción 1.

1. ¿Qué es una red? ¿Cuál es el principal objetivo para construir una red?

   Una red es un conjunto de dispositivos interconectados con el objetivo de compartir recursos, como información o servicios. Los componentes de la red deben interactuar y combinarse a través de reglas.

   Por otro lado, un sistema de comunicación está compuesto por el conjunto de computadores, software de red, medios y dispositivos de interconexión.

2. ¿Qué es Internet? Describa los principales componentes que permiten su funcionamiento.

   La red pública Internet es una red de computadoras específica que interconecta muchos dispositivos informáticos a lo largo del mundo. Su objetivo es conectar a los sistemas terminales entre sí

   Los routers y switches de la capa de enlace son los tipos de conmutadores más utilizados actualmente. Su función es reenviar paquetes hacia sus destinos finales.

   La ruta (a través de la red) es la secuencia de enlaces de comunicaciones y conmutadores de paquetes que atraviesa un paquete desde el sistema terminal emisor hasta el sistema terminal receptor.

   Las redes de conmutación de paquetes son aquellas que transportan paquetes.

   Los sistemas terminales acceden a Internet a través de los ISP (Internet Service Provider) que puede ser residencial. corporativo, universitario, de acceso inalámbrico, y los de datos móviles. Cada ISP es una red de conmutadores de paquetes y enlaces de comunicaciones y proporcionan una amplia variedad de tipos de acceso a red de los sistemas terminales. También le proporcionan el acceso a Internet a los proveedores de contenido. Los ISP tienen que estar interconectados entre ellos si van a proporcionar acceso a Internet para los sistemas terminales.

   Internet como una infraestructura que permite proporcionar servicios a aplicaciones distribuidas.

   Internet también podría definirrse como una infraestructura que proporciona servicios a las aplicaciones.

3. ¿Qué son las RFCs?

   Los documentos RFC (Request For Comments) definen protocolos tales como TCp, IP, HTTP y SMTP. Actualmente existen más de 7000 documentos RFC.

4. ¿Qué es un protocolo?

   Cuando hablamos de protocolo nos referimos al conjunto de conductas y normas a conocer, respetar y cumplir no sólo en el medio oficial ya establecido, sino también en el medio social, laboral, etc. El protocolo define el formato, el orden de los mensajes intercambiados y las acciones que se llevan a cabo en la transmisión y/o recepción de un mensaje u otro evento.

   El protocolo de red específicamente se refiere al conjunto de reglas que especifican el intercambio de datos u órdenes durante la comunicación entre las entidades que forman parte de una red. Permite la comunicación y están implementados en las componentes.

Un protocolo define el formato de los mensajes intercambiados entre dos o más entidades que se comunican, así como las acciones tomadas al producirse la transmisión y/o recepción de un mensaje u otro suceso.

Los sistemas terminales, los conmutadores de paquetes y otros dispositivos de Internet ejecutan protocolos que controlan el envío y la recepción de información dentro de Internet. El Protocolo TCP (Transmission COntrol Protocol), y el protocolo IP (Internet Protocol) son dos de los protocolos más importantes de Internet. El protocolo IP especifica el formato de los paquetes que se envían y reciben entre los routers y sistemas terminales.. **Los principales protocolos de Internet se conocen colectivamente como los protocolos TCP/IP**.

5. ¿Por qué dos máquinas con distintos sistemas operativos pueden formar parte de una misma red?

Esto es posible gracias a los protocolos estandarizados.

Dos máquinas con diferentes SO pueden coexistir ya que implementan el mismo protocolo, aunque la lógica interna la maneja cada uno de ellos.

6. ¿Cuáles son las 2 categorías en las que pueden clasificarse a los sistemas finales o End Systems? Dé un ejemplo del rol de cada uno en alguna aplicación distribuida que corra sobre Internet.

Los sistemas terminales o *host* son aquellos dispositivos que se conectan a internet y envían o reciben datos. Pueden ser tradicionales (como la PC de escritorio, estaciones de trabajo Linux y los servidores), o no tradicionales (como televisores, computadores portátiles, consolas de juegos, teléfonos móviles, cámaras web, sistemas de detección de medioambientes y automóviles y dispositivos de seguridad y electrodomésticos). Se comunican entre sí mediante una red de enlaces de comunicaciones y dispositivos de conmutación de paquetes. Se denominan terminales porque se sitúan en la frontera de red.

Las aplicaciones de internet se ejecutan sobre sistemas terminales.

Los host se clasifican en dos categorías: cliente y servidor. Los clientes suelen ser las computadoras de escritorio y portátiles, las PDA, etc, mientras que los servidores suelen ser equipos más potentes que almacenan y distribuyen páginas web, flujos de vídeo, correo electrónico, etc. Un programa cliente es aquel que se ejecuta en un sistema terminal y solicita y recibe un servicio de un programa servidor que se ejecuta en otro sistema terminal. Como un programa cliente se suele ejecutar en una computadora y el programa servidor en otra, se dice que son aplicaciones distribuidas. Estos dos programas interactúan enviándose mensajes entre sí a traves de Internet.

7. ¿Cuál es la diferencia entre una red conmutada de paquetes de una red conmutada de circuitos?

Una conmutación de paquetes es más eficiente ya que, cuando hay poco tráfico, los paquetes fluyen casi sin retardo.

La conmutación de paquetes puede tener el mismo rendimiento que la de circuitos, pero permitiendo que haya mucho más tráfico que el segundo.

Otra diferencia es la forma de compartir la velocidad de transmisión de enlace entre varios flujos de datos. La conmutación de circuitos preasigna el uso del

enlace de transmisión independiente de la demanda, con lo que el tiempo de enlace asignado innecesariamente se desperdicia.

8. Analice qué tipo de red es una red de telefonía y qué tipo de red es Internet.

Red de telefonía: red por conmutación de circuitos, canal exclusivo, eficiencia baja, calidad baja

Red de Internet: red por conmutación de paquetes, canal compartido, eficiencia alta, calidad variable

9. Describa brevemente las distintas alternativas que conoce para acceder a Internet en su hogar.

10. ¿Qué ventajas tiene una implementación basada en capas o niveles?

La implementación de un modelo por capas permite la delegación y repartición de ciertas tareas, desglosando un problema en partes más sencillas que luego se pueden unir para formar "un todo".

Cada capa, combinada con las capas que tiene por debajo, implementa una cierta funcionalidad, un cierto servicio.

Una arquitectura de capas nos permite estudiar una parte específica y bien definida de un sistema más grande y complejo, proporcionando modularidad y facilitando la modificación de la implementación del servicio suministrado por la capa. Mientras que la capa proporciona el mismo servicio a la capa que tiene por encima de ella y emplea los mismos servicios que la capa que tiene por debajo, el resto del sistema permanece invariable cuando se modifique la implementación de una capa.

11. ¿Cómo se llama la PDU de cada una de las siguientes capas: Aplicación, Transporte, Red y Enlace?

PDU, o Protocol Data Unit, son las unidades que se utilizan para el intercambio de datos entre unidades disparejas. La capa de aplicación usa datos (mensaje), la de transporte usa segmentos, la de red utiliza los paquetes (datagramas) y la de enlace utiliza frames o tramas.

12. ¿Qué es la encapsulación? Si una capa realiza la encapsulación de datos, ¿qué capa del nodo receptor realizará el proceso inverso?

La **encapsulación** es el proceso mediante el cual, al bajar por las capas del modelo TCP/IP, **cada capa agrega su propia información de control (cabeceras, y a veces trailers) a los datos**. En el nodo receptor ocurre el proceso inverso, llamado **desencapsulación**: La **misma capa** que encapsula en el emisor es la que **desencapsula en el receptor**.

13. Describa cuáles son las funciones de cada una de las capas del stack TCP/IP o protocolo de Internet. UNIDAD 1.5

Una arquitectura en capas nos permite estudiar una parte específica y bien definida de un sistema más grande y complejo. Esta simplificación por sí misma tiene un valor conside rable al proporcionar modularidad, haciendo mucho más fácil modificar la

implementación del servicio suministrado por la capa. Dado que la capa proporciona el mismo servicio a la capa que tiene por encima de ella y emplea los mismos servicios de la capa que tiene por debajo, el resto del sistema permanece invariable cuando se modifica la implementación de una capa. Estamos interesados en los servicios que ofrece una capa a la capa que tiene por encima, lo que se denomina modelo de servicio de capa.

- Aplicación: La capa de aplicación es donde residen las aplicaciones de red y sus protocolos.
- Transporte: La capa de transporte de Internet transporta los mensajes de la capa de aplicación entre los puntos terminales de la aplicación. TCPofrece a sus aplicaciones un servicio orientado a la conexión. Este servicio propor ciona un suministro garantizado de los mensajes de la capa de aplicación al destino y un mecanismo de control del flujo (es decir, adaptación de las velocidades del emisor y el receptor). TCP también divide los mensajes largos en segmentos más cortos y proporciona un mecanismo de control de congestión, de manera que un emisor regula su velocidad de transmisión cuando la red está congestionada.
- Red: La capa de red de Internet es responsable de trasladar los paquetes de la capa de red, conoci dos como datagramas, de un host a otro. El protocolo de la capa de transporte (TCP o UDP) de Internet de un host de origen pasa un segmento de la capa de transporte y una dirección de destino a la capa de red, al igual que damos al servicio de correo postal una carta con una dirección de destino. Luego, la capa de red proporciona el servicio de suministrar el seg mento a la capa de transporte del host de destino.
- Enlace: La capa de red de Internet encamina un datagrama a través de una serie de routers entre el origen y el destino. Para trasladar un paquete de un nodo (host o router) al siguiente nodo de la ruta, la capa de red confía en los servicios de la capa de enlace.
- Física: mover los bits individuales dentro de la trama de un nodo al siguiente. Los protocolos de esta capa son de nuevo dependientes del enlace y, por tanto, dependen del medio de transmisión del enlace (por ejemplo, cable de cobre de par trenzado o fibra óptica monomodo).

14. Compare el modelo OSI con la implementación TCP/IP.

	OSI	TCP/IP
se divide en capas	SI	NO
tiene capa de aplicación (diferente)	SI	SI
tiene capa de transporte similar	SI	SI
tiene capa de red	NO	SI
tiene tecnología de conmutación de paquetes	SI	SI
estándares a través de los cuales se desarrolló	NO	SI

Internet		
modelo simple	NO	SI

Práctica 2

Redes y Comunicaciones LINTI - UNLP Práctica 2 - Capa de Aplicación HTTP
96
Introducción

1. ¿Cuál es la función de la capa de aplicación? 46

   La capa de aplicación es donde residen las aplicaciones de red y sus protocolos, como HTTP, SMTP y FTP. Un protocolo de la capa de aplicación está distribuído a lo largo de varios sistemas terminales, estando la aplicación en un sistema terminal que utiliza el protocolo para intercambiar paquetes de información con la aplicación de otro sistema terminal. A este paquete de información de la capa de aplicación se lo denomina **mensaje**.

2. Si dos procesos deben comunicarse:
   a. ¿Cómo podrían hacerlo si están en diferentes máquinas?

      Si los procesos están en diferentes máquinas, para poder comunicarse, necesitan conectarse a Internet e intercambiar información siguiendo los protocolos. El protocolo de transporte TCP ofrece a sus aplicaciones un servicio orientado a conexión, divide los mensajes en segmentos y proporciona también un mecanismo de control de flujo.

   b. Y si están en la misma máquina, ¿qué alternativas existen?

      Si están en la misma máquina, los procesos pueden utilizar el protocolo **UDP** que proporciona a sus aplicaciones un servicio sin conexión. Es un servicio básico que no ofrece ninguna fiabilidad, ni control de flujo ni control de congestión.

3. Explique brevemente cómo es el modelo Cliente/Servidor. Dé un ejemplo de un sistema Cliente/Servidor en la “vida cotidiana” y un ejemplo de un sistema informático que siga el modelo Cliente/Servidor. ¿Conoce algún otro modelo de comunicación?

   Un programa cliente es un programa que se ejecuta en un sistema terminal que solicita y recibe de un programa servidor que se ejecuta en otro sistema terminal. Se dice que son aplicaciones distribuídas porque, por lo general, el programa cliente se ejecuta en un sistema terminal y el programa servidor en otro. Estos dos programas interactúan enviándose mensajes a través de Internet.

   Por ejemplo, a la hora de realizar una transferencia bancaria, el dispositivo del usuario que contiene la aplicación “Cuenta DNI” (cliente), solicita al Banco Nación (servidor) conexión. Una vez aceptada, el cliente servidor solicita los datos de la transferencia, como cuenta origen y cuenta destino, importe, etc. El sistema cliente envía dichos datos y espera recibir el mensaje de transferencia exitosa.

   Otro ejemplo es la Web, en la que un servidor web siempre activo sirve las solicitudes de los navegadores que se ejecutan en los hosts clientes. Cuando un servidor web recibe una solicitud de un objeto de un host cliente, responde enviándole el objeto solicitado.

   Si miramos otros modelos de comunicación, en la **arquitectura P2P** los host actúan como servidor y cliente a la vez, y existe una mínima dependencia de una infraestructura de servidores siempre activos. En su lugar, la aplicación explota la comunicación directa entre parejas de host conectados de forma intermitente, conocidos como peers. Esta arquitectura se utiliza en muchas de las aplicaciones

actuales más populares. Por otro lado, existen aplicaciones con **arquitecturas híbridas** que combinan elementos cliente-servidor y P2P. Por ejemplo, en aplicaciones de mensajería instantánea los servidores se utilizan para hacer un seguimiento de las direcciones IP de los usuarios, pero los mensajes de usuario a usuario se envían directamente entre los hosts.

4. Describa la funcionalidad de la entidad genérica “Agente de usuario” o “User agent”.
   El agente de usuario especifica el tipo de navegador que está haciendo la solicitud al servidor. Esto es útil porque el servidor puede enviar versiones diferentes del mismo objeto a los distintos tipos de agentes de usuario (el mismo URL direcciona a cada una de las versiones).

5. ¿Qué son y en qué se diferencian HTML y HTTP?
   Tanto HTML como HTTP son protocolos, pero HTL establece un estándar para los formatos de documentos, y HTTP es el protocolo de la capa de aplicación de la WEb, que define el formato y la secuencia de los mensajes que se pasan entre el navegador web y el servidor web.

6. HTTP tiene definido un formato de mensaje para los requerimientos y las respuestas. (Ayuda: apartado “Formato de mensaje HTTP”, Kurose). Página 101
   a. ¿Qué información de la capa de aplicación nos indica si un mensaje es de requerimiento o de respuesta para HTTP? ¿Cómo está compuesta dicha información?¿Para qué sirven las cabeceras? 1 Redes y Comunicaciones LINTI - UNLP
   b. ¿Cuál es su formato? (Ayuda: https://developer.mozilla.org/es/docs/Web/HTTP/Headers)

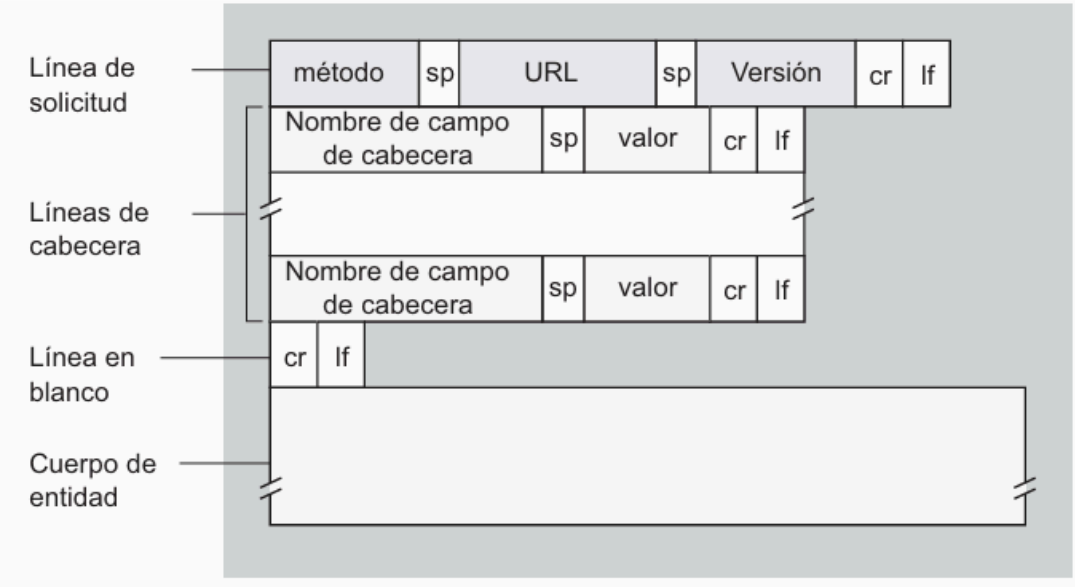
      REQUERIMIENTOS

      La primera línea se denomina **línea de solicitud** y las siguientes son **líneas de cabecera**. La línea de solicitud tiene tres campos: el campo de método (GET, POST, HEAD, PUT, DELETE), el campo URL y el campo de la versión HTTP. La mayoría de los mensajes de solicitud HTTP usan el método GET. Este método se emplea cuando el navegador solicita un objeto, identificando dicho objeto en el campo URL.

      En las líneas de cabecera tenemos

      - Host: especifica el host en el que reside el objeto. Esta línea es necesaria porque las cachés proxy web necesitan la información proporcionada por la línea de cabecera del host.
      - Connection: close - el navegador está diciendo al servidor que no desea molestarse en trabajar con conexiones persistentes, sino que desea que el servidor cierre la conexión después de enviar el objeto solicitado.
      - User-agent: especifica el agente de usuario, es decir, el tipo de navegador que está haciendo la solicitud al servidor. Esto es útil porque el servidor puede enviar versiones diferentes del mismo objeto a los distintos tipos de agentes de usuario (el mismo URL direcciona a cada una de las versiones).

- Accept-language -> es una de las cabeceras de negociación del contenido disponibles en HTTP



- Cuerpo de entidad: este campo queda vacío cuando se utiliza el método GET, pero no cuando se usa el método POST. Un cliente HTTP suele usar el método POST cuando el usuario completa un formulario. Con el mensaje post, el usuario solicita una página web al servidor, pero el contenido concreto de la misma dependerá de lo que el usuario haya escrito en los campos del formulario. Este cuerpo de entidad contendrá lo que el usuario haya introducido en los campos del formulario. A veces los formularios HTML usan el método GET e incluyen los datos de entrada, especificados en los campos del formulario, en la URL solicitada. Por ejemplo, si un formulario emplea el método GET y tiene dos campos, y las entradas a esos dos campos son monkeys y bananas, entonces el URL tendrá la estructura www.unsitio.com/animalsearch?monkeys&bananas.

  El método HEAD es similar al GET. Cuando el servidor recibe una solicitud con el método HEAD, responde con un mensaje HTTP pero excluye el objeto solicitado. Se suele usar para labores de depuración.

  El método PUT se usa junto con herramientas de publicación WEB. Esto permite a un usuario cargar un objeto en la ruta específica (directorio) en un servidor web determinado. También lo usan las aplicaciones que necesitan cargar objetos en servidores web.

  El método DELETE permite a un usuario o a una aplicación borrar un objeto de un servidor web.

RESPUESTAS

c. Suponga que desea enviar un requerimiento con la versión de HTTP 1.1 desde curl/7.74.0 a un sitio de ejemplo como www.misitio.com para obtener el recurso /index.html. En base a lo indicado, ¿qué información debería enviarse mediante encabezados? Indique cómo quedaría el requerimiento.

   Mi solución
   GET /indez.html HTTP 1.1
   Host: www.curl/7.74.0.com
   Connection: close
   User-agent: Ecosia
   Accept.language: ???

   Solución de la IA
   GET /index.html HTTP/1.1
   Host: www.misitio.com
   User-Agent: curl/7.74.0
   Accept: */*
   Connection: close

7. Utilizando la VM, abra una terminal e investigue sobre el comando curl. Analice para qué sirven los siguientes parámetros (-I, -H, -X, -s).
8. Ejecute el comando curl sin ningún parámetro adicional y acceda a www.redes.unlp.edu.ar. Luego responda:
   a. ¿Cuántos requerimientos realizó y qué recibió? Pruebe redirigiendo la salida (>) del comando curl a un archivo con extensión html y abrirlo con un navegador.
   b. ¿Cómo funcionan los atributos href de los tags link e img en html?
   c. Para visualizar la página completa con imágenes como en un navegador, ¿alcanza con realizar un único requerimiento?
   d. ¿Cuántos requerimientos serían necesarios para obtener una página que tiene dos CSS, dos Javascript y tres imágenes? Diferencie cómo funcionaría un navegador respecto al comando curl ejecutado previamente.
9. Ejecute a continuación los siguientes comandos: curl -v -s www.redes.unlp.edu.ar > /dev/null curl -I -v -s www.redes.unlp.edu.ar
   a. ¿Qué diferencias nota entre cada uno?
   b. ¿Qué ocurre si en el primer comando se quita la redirección a /dev/null? ¿Por qué no es necesaria en el segundo comando?
   c. ¿Cuántas cabeceras viajaron en el requerimiento? ¿Y en la respuesta?
10. ¿Qué indica la cabecera Date?
11. En HTTP/1.0, ¿cómo sabe el cliente que ya recibió todo el objeto solicitado de manera completa? ¿Y en HTTP/1.1?
12. Investigue los distintos tipos de códigos de retorno de un servidor web y su significado. Considere que los mismos se clasifican en categorías (2XX, 3XX, 4XX, 5XX).
13. Utilizando curl, realice un requerimiento con el método HEAD al sitio www.redes.unlp.edu.ar e indique:
    a. ¿Qué información brinda la primera línea de la respuesta?
    b. ¿Cuántos encabezados muestra la respuesta?

c. ¿Qué servidor web está sirviendo la página? 2 Redes y Comunicaciones LINTI - UNLP
d. ¿El acceso a la página solicitada fue exitoso o no?
e. ¿Cuándo fue la última vez que se modificó la página?
f. Solicite la página nuevamente con curl usando GET, pero esta vez indique que quiere obtenerla sólo si la misma fue modificada en una fecha posterior a la que efectivamente fue modificada. ¿Cómo lo hace? ¿Qué resultado obtuvo? ¿Puede explicar para qué sirve?

14. Utilizando curl, acceda al sitio www.redes.unlp.edu.ar/restringido/index.php y siga las instrucciones y las pistas que vaya recibiendo hasta obtener la respuesta final. Será de utilidad para resolver este ejercicio poder analizar tanto el contenido de cada página como los encabezados.
15. Utilizando la VM, realice las siguientes pruebas:
    a. Ejecute el comando 'curl www.redes.unlp.edu.ar/extras/prueba-http-1-0.txt' y copie la salida completa (incluyendo los dos saltos de línea del final).
    b. Desde la consola ejecute el comando telnet www.redes.unlp.edu.ar 80 y luego pegue el contenido que tiene almacenado en el portapapeles. ¿Qué ocurre luego de hacerlo?
    c. Repita el proceso anterior, pero copiando la salida del recurso /extras/prueba-http-1-1.txt. Verifique que debería poder pegar varias veces el mismo contenido sin tener que ejecutar el comando telnet nuevamente.
16. En base a lo obtenido en el ejercicio anterior, responda:
    a. ¿Qué está haciendo al ejecutar el comando telnet?
    b. ¿Qué método HTTP utilizó? ¿Qué recurso solicitó?
    c. ¿Qué diferencias notó entre los dos casos? ¿Puede explicar por qué?
    d. ¿Cuál de los dos casos le parece más eficiente? Piense en el ejercicio donde analizó la cantidad de requerimientos necesarios para obtener una página con estilos, javascripts e imágenes. El caso elegido, ¿puede traer asociado algún problema?
17. En el siguiente ejercicio veremos la diferencia entre los métodos POST y GET. Para ello, será necesario utilizar la VM y la herramienta Wireshark. Antes de iniciar considere:
    ■ Capture los paquetes utilizando la interfaz con IP 172.28.0.1. (Menú "Capture ->Options". Luego seleccione la interfaz correspondiente y presione Start).
    ■ Para que el analizador de red sólo nos muestre los mensajes del protocolo http introduciremos la cadena 'http' (sin las comillas) en la ventana de especificación de filtros de visualización (display-filter). Si no hiciéramos esto veríamos todo el tráfico que es capaz de capturar nuestra placa de red. De los paquetes que son capturados, aquel que esté seleccionado será mostrado en forma detallada en la 3 Redes y Comunicaciones LINTI - UNLP sección que está justo debajo. Como sólo estamos interesados en http ocultaremos toda la información que no es relevante para esta práctica (Información de trama, Ethernet, IP y TCP). Desplegar la información correspondiente al protocolo HTTP bajo la leyenda "Hypertext Transfer Protocol".
    ■ Para borrar la cache del navegador, deberá ir al menú "Herramientas->Borrar historial reciente". Alternativamente puede utilizar Ctrl+F5 en el navegador para forzar la petición HTTP evitando el uso de caché del navegador.

■ En caso de querer ver de forma simplificada el contenido de una comunicación http, utilice el botón derecho sobre un paquete HTTP perteneciente al flujo capturado y seleccione la opción Follow TCP Stream.

a. Abra un navegador e ingrese a la URL: www.redes.unlp.edu.ar e ingrese al link en la sección “Capa de Aplicación” llamado “Métodos HTTP”. En la página mostrada se visualizan dos nuevos links llamados: Método GET y Método POST. Ambos muestran un formulario como el siguiente:
b. Analice el código HTML
c. Utilizando el analizador de paquetes Wireshark capture los paquetes enviados y recibidos al presionar el botón Enviar.
d. ¿Qué diferencias detectó en los mensajes enviados por el cliente?
e. ¿Observó alguna diferencia en el browser si se utiliza un mensaje u otro?

18. Investigue cuál es el principal uso que se le da a las cabeceras Set-Cookie y Cookie en HTTP y qué relación tienen con el funcionamiento del protocolo HTTP.
19. ¿Cuál es la diferencia entre un protocolo binario y uno basado en texto? ¿De qué tipo de protocolo se trata HTTP/1.0, HTTP/1.1 y HTTP/2?
20. Responder las siguientes preguntas:
    a. ¿Qué función cumple la cabecera Host en HTTP 1.1? ¿Existía en HTTP 1.0? ¿Qué sucede en HTTP/2? (Ayuda: 4 Redes y Comunicaciones LINTI - UNLP https://undertow.io/blog/2015/04/27/An-in-depth-overview-of-HTTP2.html para HTTP/2)
    b. En HTTP/1.1, ¿es correcto el siguiente requerimiento? GET /index.php HTTP/1.1 User-Agent: curl/7.54.0
    c. ¿Cómo quedaría en HTTP/2 el siguiente pedido realizado en HTTP/1.1 si se está usando https? GET /index.php HTTP/1.1 Host: www.info.unlp.edu.ar

Ejercicio de Parcial

```
curl -X ?? www.redes.unlp.edu.ar/??
> HEAD /metodos/ HTTP/??
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.54.0

< HTTP/?? 200 OK
< Server: nginx/1.4.6 (Ubuntu)
< Date: Wed, 31 Jan 2018 22:22:22 GMT
< Last-Modified: Sat, 20 Jan 2018 13:02:41 GMT
< Content-Type: text/html; charset=UTF-8
< Connection: close
```

a. ¿Qué versión de HTTP podría estar utilizando el servidor?
b. ¿Qué método está utilizando? Dicho método, ¿retorna el recurso completo solicitado?
c. ¿Cuál es el recurso solicitado?
d. ¿El método funcionó correctamente?
e. Si la solicitud hubiera llevado un encabezado que diga: If-Modified-Since: Sat, 20 Jan 2018 13:02:41 GMT ¿Cuál habría sido la respuesta del servidor web? ¿Qué habría hecho el navegador en este caso?